SMART INDIA HACKATHON 2026 · SOFTWARE

Team Handbook

Integrated Dashboard for Monitoring and Tracking Institution-Related Complaint Cases Across AICTE Bureaus/Cells

Problem Statement ID 26264 · Organisation: AICTE
Working project title: **CaseNexus** (a placeholder; the team may rename it)

What this document is

A single place where every teammate can learn the problem, the theory behind our approach, the workflow, the pipelines, the technology and the plan. It is written for people who have **not** yet read the problem statement. Read Sections 1 to 4 first; then read the sections for your role.

Companion file	Use it for
SIH2026_PPT_Content.md	Paste-ready text for the 6-slide submission, diagram briefs, speaker notes, judge Q and A
AICTE_Complaint_Dashboard_Guide.md	Full design document (features, architecture, decisions)
AICTE_Prototype_Build_Guide.md	Stage-by-stage build manual with schema and starter code

Version 1 · 23 October 2026 · Living document. Items marked **[OPEN]** still need a team decision. References were checked on the date above; library versions, prices and portal rules can change.

Contents

1. The problem in 60 seconds 3
2. Context: AICTE and grievance handling 3
3. The problem statement, decoded 4
4. Our idea 5
5. Theory primer (explained from scratch) 6
6. The overlap engine in detail 8
7. Workflow and roles 10
8. Architecture and data pipelines 11
9. Online / Offline mode and privacy 12
10. Technology stack and why 13
11. Data model 14
12. Prototype plan 14
13. Evaluation plan 15
14. Risks and open decisions 16
15. Seven-minute demo script 16
16. Submitting the idea PPT 17
17. Glossary 18
18. References 18

Reading order by role

Role	Read first	Then
Everyone	1, 2, 3, 4	7 (workflow), 14 (risks), 17 (glossary)
Frontend	8 (architecture), 10 (stack)	11 (data model), 15 (demo)
Backend and database	8, 9, 11	10, 12 (stages)
ML and matching	5 (theory), 6 (engine)	13 (evaluation), 11
PPT and presentation	16 (submission), 3, 4	15 (demo), 17, 18 (references)
DevOps and QA	9 (modes), 12	13, 14

1. The problem in 60 seconds

AICTE (All India Council for Technical Education) approves and regulates technical institutions in India. People complain to it about colleges: fake faculty, excess fees, unapproved courses, withheld certificates, weak infrastructure, false claims of approval. Different internal units handle those complaints.

Unit	What it does
RB (Regulation Bureau)	Takes regulatory and enforcement action against institutions
PGRC (Public Grievance Redressal Cell)	Receives and resolves grievances from students, faculty and the public
Other Bureaus/Cells	Handle other functions that may also touch the same institution
Regional Offices/Centres	Work with institutions in their region

The problem: each unit keeps its own records. Nobody can see, in one place and in real time, that *the same institution* is being handled by several units. The result is duplicated work, conflicting actions, delays, and no clear answer to the question **"who is dealing with this institution right now?"**

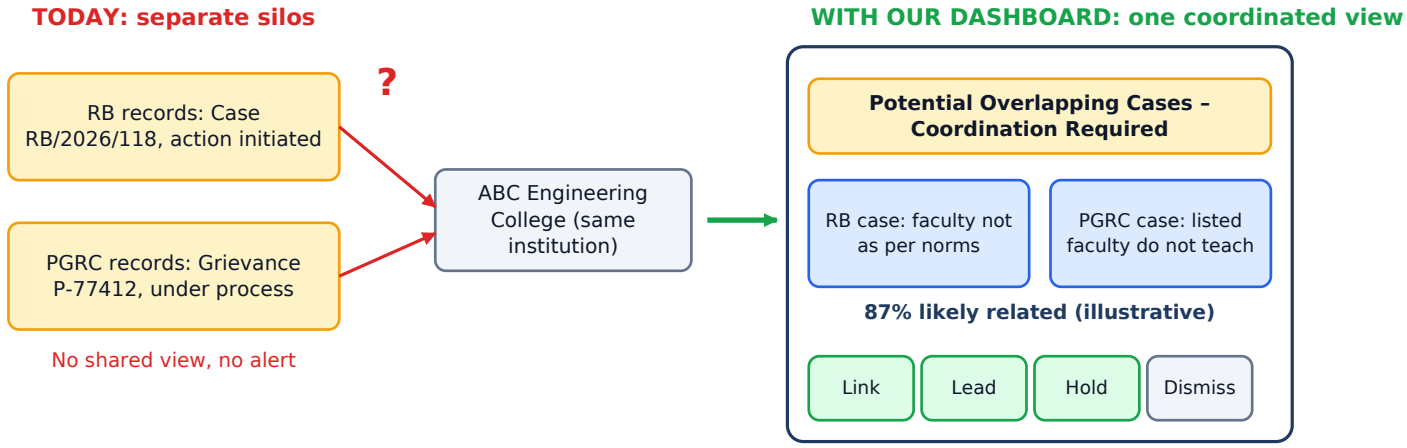


Figure 1. Today versus our coordinated dashboard (numbers are illustrative).

One sentence to remember: we are not only building a dashboard; we are building a **coordination layer** that notices when two units are working on the same institution, explains why, and helps them decide who leads.

2. Context: AICTE and grievance handling

These facts were read from official pages on 23 October 2026 and are the only external facts about AICTE we rely on.

- AICTE's own grievance page describes the **Public Grievance Redressal Cell (PGRC)**. It deals with grievances and complaints from students, faculty and stakeholders of AICTE-approved institutions, ragging complaints, complaints that institutions violate norms and standards, and grievances that arrive through the **CPGRAMS** portal. https://aicte.gov.in/node/422
- **CPGRAMS** is the Government of India's central grievance portal. A PIB document (December 2024) reports that it connects 92 Central Ministries/Departments/Organisations with 36 States/UTs, resolved about 7.0 million grievances between 2022 and 2024, and that the grievance redressal timeline was cut from 30 days to 21 days. These are government self-reported figures. https://static.pib.gov.in/WriteReadData/specificdocs/documents/2024/dec/doc20241230478501.pdf

What we do not know: AICTE's own internal timelines per complaint category, the formats of the RB and PGRC systems, and how cases are numbered. The problem statement does not say. We therefore make these **configurable** (deadlines) or **adapter-based** (data import), and we say so openly in the PPT.

3. The problem statement, decoded

Official title: Integrated Dashboard for Monitoring and Tracking Institution-Related Complaint Cases Across AICTE Bureaus/Cells (PS ID 26264).

3.1 Objectives (what the dashboard must let officials do)

- View all institution-related complaint/grievance cases in one place
- Identify the Bureau/Cell currently handling each case
- Track the complete status and movement of a case
- Identify cases where RB and PGRC actions overlap, or where several Bureaus/Cells deal with the same institution
- Avoid duplication and ensure coordination between Bureaus/Cells
- Monitor pending, disposed, under-process, transferred and escalated cases
- Generate institution-wise, region-wise, Bureau/Cell-wise and status-wise reports
- Alert on pending cases and cases approaching prescribed timelines

3.2 The eight required feature areas, and how we cover each

#	PS feature area	What the PS lists	Our plan
1	Centralized institution profile	Unique ID / AICTE Permanent ID; name, address, region; consolidated complaint history	Institution table keyed on AICTE ID; 360 page listing all cases from all units
2	Complaint/case tracking	Case ID, receipt date, category, status, Bureau/Cell, officer, last action, next action, date of last action, expected disposal date	Case table with all those fields; SLA due date calculated from category
3	Cross-Bureau/Cell overlap detection	Auto-identify same-institution cases across RB, PGRC and others; alert; link related cases	Hybrid overlap engine, High/Medium/Info levels, link/lead/hold/dismiss
4	Real-time dashboard	Totals; pending; under process; disposed; transferred; beyond timeline; institution-wise status; Bureau/Cell workload	KPI cards and charts that refresh every few seconds
5	Workflow and action history	Chronological history; responsible officer/Bureau; timestamp; remarks; documents; transfer/escalation history	Action table rendered as a timeline; documents in object storage
6	Search and filters	Institution name, AICTE ID, case ID, Bureau/Cell, region, category, status, date range	Filter bar plus fuzzy name search and full-text search
7	Alerts and notifications	Duplicate/overlap alerts; pending reminders; escalation; timeline/SLA alerts	In-app alerts center; scheduled SLA scan; email on the online route
8	Reports and analytics	Bureau-wise; region-wise; institution history; ageing; disposal rate; overlaps; monthly/quarterly	Report pages with Excel/PDF export

3.3 The innovation the PS explicitly asks for

Use **data matching, rule-based logic and/or AI-assisted analytics** to identify complaints about the same institution *even when the descriptions or case references are different*. The PS's own example:

Institution A	Complaint received by PGRC	Case under process
Institution A	Complaint received by RB	Action initiated

The system should detect the common institution and show: **"Potential Overlapping Cases – Coordination Required"**. This is the heart of our project and of the evaluation.

3.4 Expected outcomes and users

Expected outcomes	Target users
Single source of truth; less duplicated work; no conflicting parallel actions; better RB/PGRC coordination; timely disposal; accountability and transparency; real-time monitoring; data-driven decisions	RB; PGRC; other Bureaus/Cells; Regional Offices/Centres; authorised officers and senior management; Competent Authority / AICTE administration

3.5 Six hidden hard problems behind the statement

Hard problem	Why it matters	Our answer
Which institution is this?	Complaints often miss or misspell the ID; names vary ("Govt. Engg. College")	ID first; then fuzzy, rarity-weighted name matching; low confidence goes to a human
What counts as an overlap?	Same institution but different issues is not a duplicate; treating it as one floods officers with false alerts	Three levels (High/Medium/Info) and a learned score
Unknown source systems	The PS does not describe RB/PGRC data formats	Import adapter (CSV/API) into one common schema
Unknown deadlines	Timelines differ by category and are not given	Configurable SLA rules with a source reference; data-driven risk warnings
Sensitive data	Complainants and whistleblowers need protection	Role-based access, masking, audit log, per-case offline processing
Adoption	Officers will not enter data twice	Single window, pre-action banner, minimal extra entry

4. Our idea

Working title: CaseNexus. Tagline: "*No AICTE officer should act on an institution blind again.*"

4.1 Three layers of value

Layer	Meaning	Required by PS?
1. Visibility	One place to see all cases, owners, status, history	Yes (basic)
2. Intelligence	Detect that cases relate even when wording and IDs differ; explain why	Yes (innovation clause)
3. Coordination	Link cases, nominate a lead Bureau/Cell, hold parallel action, record decisions	Implied by 'avoid duplication and ensure coordination'; our differentiator

4.2 What makes it different from a plain dashboard

1. **Two-question engine.** 'Is it the same institution?' and 'Is it the same issue?' are answered separately, then combined.
2. **Explainable.** Each alert shows which signals pushed the score up or down.
3. **Calibrated, learned thresholds.** The 'High' level is set to reach a target precision so officers are not flooded with false alarms. Every parameter is recorded in a registry with its origin.
4. **Coordination actions.** Link, nominate lead, hold parallel action, dismiss with reason; dismissals become training labels.
5. **Privacy switch.** An Online/Offline mode and a per-case 'sensitive' flag that forces local processing.
6. **Multilingual.** The text model supports 50 languages, so Hindi and other Indian-language complaints are handled.

4.3 A worked example

When	What happens	System response
Day 1	A student complains to PGRC: faculty listed on the website of ABC Engineering College do not teach there	Case P-1 created; institution resolved by AICTE ID; no other active cases; no alert
Day 7	RB registers an inspection finding: teaching staff at ABC College not as per norms	Case R-1 created; embedding compared with P-1; similar meaning, same institution, different Bureaus, both open
Day 7 (seconds later)	-	**High overlap** alert to PGRC and RB: 'Potential Overlapping Cases – Coordination Required', with the score breakdown
Day 7	Officers review side by side; RB nominates itself lead; PGRC case gets a 'hold parallel action' flag	Both timelines record the link; decision logged in the audit trail
Day 40	Another PGRC complaint about ABC College's fees arrives	Different issue: shown as **Medium** or Info, not as a High duplicate

5. Theory primer (explained from scratch)

This section explains every technique we use, in the order a newcomer needs them. Numbers marked 'illustrative' are teaching examples, not results.

5.1 Entity resolution: is this the same institution?

Entity resolution (also called record linkage) decides whether two records refer to the same real-world thing. For us the things are institutions. We use a ladder of methods, from cheap and exact to flexible:

Rung	Method	Example	Where used
1	Exact key	AICTE Permanent ID equals	First check; confidence 1.0
2	String similarity (trigrams)	'government engineering college pune' vs 'govt engg college pune'	PostgreSQL **pg_trgm** breaks text into 3-character pieces and compares how many are shared
3	Rarity-weighted tokens (IDF)	The word 'ambedkar' tells us more than 'college'	Our resolution code weights each word by how rare it is among institutions
4	Probabilistic linkage (Fellegi-Sunter)	Estimate how strongly each field agreeing or disagreeing supports a match	Planned upgrade, via the Splink library

Rarity weighting. For each word t we compute w(t) = ln((N+1) / (df+1)) + 1, where N is the number of institutions and df the number containing t. The similarity of two names is the total weight of shared words divided by the total weight of all words in either name. Generic words ('college', 'institute') get low weight automatically, so we do not need a hand-written stop-word list.

Fellegi-Sunter in plain words. For each field, two numbers are estimated: **m** = chance the field agrees when the records truly match, and **u** = chance it agrees by coincidence for different institutions. An agreeing field adds log2(m/u) to the evidence; a disagreeing one adds log2((1-m)/(1-u)). The summed evidence is converted to a match probability. The m and u values can be estimated *without labelled data* using the EM algorithm.

Illustrative example	m	u	Evidence if field agrees
City agrees	0.95	0.01	log2(0.95/0.01) = +6.6
City disagrees	0.95	0.01	log2(0.05/0.99) = -4.3
Rare word 'ambedkar' agrees	0.95	0.002	log2(0.95/0.002) = +8.9
Common word 'college' agrees	0.95	0.6	log2(0.95/0.6) = +0.7

This is the **term-frequency adjustment**: agreeing on a rare word is strong evidence, agreeing on a common word is weak. Source: Fellegi and Sunter (1969); the Splink library implements the model with unsupervised parameter estimation and term-frequency adjustment (refs 3 and 4).

> **Be precise in the PPT:** the first prototype implements rungs 1 to 3 (ID, trigram similarity and rarity-weighted tokens). Rung 4 (full Fellegi-Sunter via Splink) is the planned upgrade. Do not claim it as built until it is.

5.2 Embeddings: is it the same issue?

An **embedding** turns a piece of text into a list of numbers (a vector) so that texts with similar *meaning* get vectors pointing in similar directions. Our model, **paraphrase-multilingual-MiniLM-L12-v2**, produces 384 numbers per text, supports 50 languages and is Apache-2.0 licensed (ref 6). We run it through **fastembed**, which uses ONNX Runtime on CPU, so no GPU is needed (ref 8).

```
cosine_similarity(a, b) = (a . b) / (|a| x |b|)      range -1 to 1; closer to 1 = more similar meaning
pgvector operator <=> returns cosine DISTANCE, so  similarity = 1 - distance
```

Why not compare texts directly with a large language model? Sentence-BERT's authors report that finding the most similar pair among 10,000 sentences took about 65 hours with a standard BERT pair-comparison setup versus about 5 seconds with sentence embeddings (ref 5). Embeddings are computed once per case and compared with cheap arithmetic.

Limit to remember: embeddings capture topical similarity. 'Fees too high' and 'refund denied' are both fee-related but legally different. This is why we add other features and test with **hard negatives** (same institution, different issue).

5.3 Combining signals: logistic regression

For each pair of cases we compute six features (Section 6.3). A logistic regression computes z = b + w1x1 + ... + w6x6 and the probability p = 1 / (1 + e^-z). Because the model is linear, each term wi x xi shows how much one feature pushed the score up or down; these are the bars in the explanation card. We chose it because it is explainable, trains in seconds on small data, and needs no GPU.

> **Honest detail:** in our design the final *probability* comes from the calibrated model, while the *bars* come from a plain logistic regression fitted on the same data. They agree in direction but are not numerically identical; the UI labels the bars as an explanation.

5.4 Calibration: making '0.9' mean 90%

Raw model scores are not always true probabilities. **Calibration** fixes this so that, among pairs scored about 0.9, roughly 90% really are related. scikit-learn's CalibratedClassifierCV offers **sigmoid** (Platt scaling, a logistic fit on the scores) and **isotonic** (non-parametric). Its documentation recommends sigmoid for very uncalibrated classifiers on very imbalanced data and warns that isotonic overfits when calibration samples are far fewer than about 1,000 (ref 7). Our early synthetic set is small, so we start with sigmoid.

5.5 Precision, recall and choosing thresholds

```
precision = TP / (TP + FP)     'of the alerts we raised, how many were right?'
recall    = TP / (TP + FN)     'of the real overlaps, how many did we catch?'
```

Most pairs of cases are unrelated, so plain accuracy is misleading (a model that says 'never related' is highly accurate and useless). We use precision, recall and PR-AUC. **High** threshold = the lowest score that still achieves the target precision (initial policy target 0.90), because false alarms cause **alert fatigue**. **Medium** threshold is set for a target recall (initial 0.90). These two targets are **policy choices**, recorded as such; the threshold values themselves are learned.

Leakage warning. If pairs from the same institution land in both the training and test sets, results look better than they are. We split **by institution**, so the test set contains institutions the model never saw.

5.6 SLA risk: warning before a deadline is missed

Deadlines are **policy** (stored per category with a source). Warnings are **data-driven**: for an open case of age a days with deadline D, we estimate P(breach) = (closed cases of the same category that took longer than D) / (closed cases that took longer than a). If fewer than 15 historical cases exist we fall back to simple deadline checks and say so. This simple estimate ignores cases still open (so it can be optimistic); a Kaplan-Meier survival estimate is the proper upgrade.

5.7 Access control, masking and audit

- **RBAC:** each user has a role (officer, regional, management, admin); regional users see only their region's institutions.
- **Masking:** complainant identity is hidden by default and revealed only to permitted roles, with an audit entry.
- **Audit log:** append-only record of decisions, mode changes, exports and sensitive views, for accountability and transparency.

5.8 Cold start: learning without real data

We have no real labelled AICTE data. We therefore (1) generate a synthetic set with known answers (paraphrased true overlaps and hard negatives), (2) label every resulting parameter *learned_synthetic*, (3) retrain on real officer confirm/dismiss decisions as they accumulate, and (4) tell judges plainly that current numbers come from synthetic data.

6. The overlap engine in detail

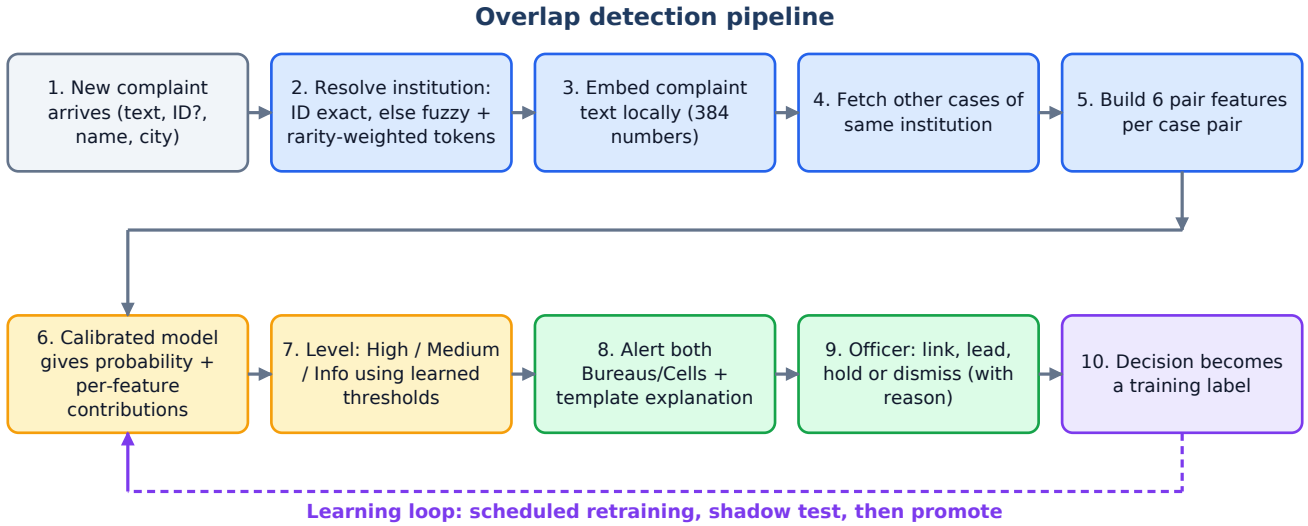


Figure 2. From a new complaint to an officer decision and back into learning.

6.1 Institution resolution

1. If an AICTE Permanent ID is given and exists, accept it (confidence 1.0).
2. Otherwise retrieve up to five candidates with pg_trgm; score each with a blend of trigram similarity and rarity-weighted token overlap, plus a small bonus if the city matches.
3. Score at or above the auto-accept threshold: accept. Between the review floor and auto-accept: accept but ask an officer to confirm. Below: ask the officer to pick manually.
4. The two thresholds start as *default_pending_data* and are tuned on about 100 labelled name variants (Section 13).

6.2 Overlap levels

Level	Meaning	Example	What the user sees
High	Same institution, same issue, handled by different Bureaus/Cells, both open	Fake faculty at PGRC and RB	'Potential Overlapping Cases – Coordination Required' to both units
Medium	Same institution, related or different issue, both open	Fee complaint at PGRC, approval violation at RB	'Related cases' notice
Info	Same institution, similar but older or closed case	Past resolved complaint	Shown in the institution history only

6.3 Features for each pair of cases

Feature	Meaning	Why it helps
inst_conf	Lower of the two institution-match confidences	Do not trust an overlap built on a shaky institution match
cos	Cosine similarity of the two complaint embeddings	Same issue, different words
same_category	1 if categories equal	Cheap strong signal
log_days	Log of days between receipt dates	Cases close in time are more likely related
both_active	1 if both are still open	Overlap matters for coordination only while action is possible
cross_bureau	1 if handled by different Bureaus/Cells	The coordination risk the PS cares about

6.4 What the officer sees

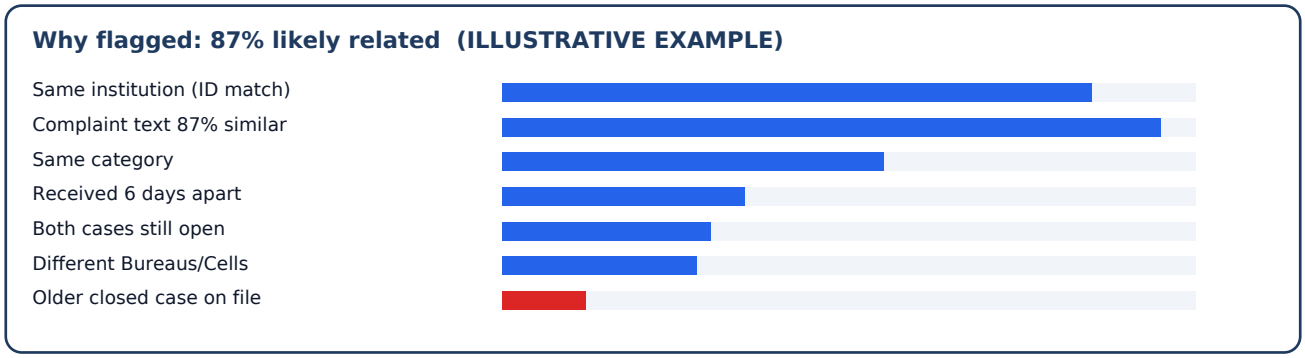


Figure 3. Example explanation card. Values are illustrative; blue bars push toward 'related', red bars push away.

6.5 Coordination actions

Action	Effect in the system
Link	Marks the overlap as confirmed; writes a 'link' entry on both case timelines
Nominate lead	Records which Bureau/Cell leads; the other becomes supporting
Hold parallel action	Sets a flag on the non-lead case so officers see action is on hold pending coordination
Dismiss (reason required)	Marks 'not related'; reason stored; becomes a training label

[OPEN] Who should be the default lead is an AICTE policy question (for example, the unit with regulatory authority). The prototype lets officers choose; do not claim the system decides.

7. Workflow and roles

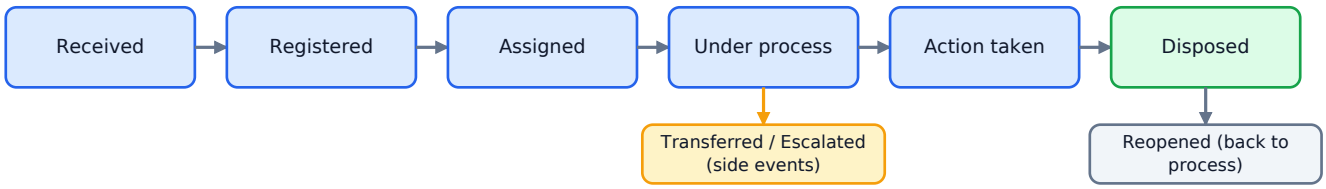


Figure 4. Case lifecycle. Transfers and escalations are events on the timeline; a disposed case can be reopened.

7.1 End-to-end flow

1. **Intake.** A complaint arrives at RB, PGRC or a Regional Office. It enters via import/sync from the source or manual registration.
2. **Institution resolution.** ID first, then fuzzy matching; low confidence goes to an officer.
3. **Case creation.** Case ID, category (classifier suggestion that the officer can change), SLA deadline, Bureau/Cell, officer; text embedding generated locally.
4. **Overlap scan.** Compare with the institution's other cases; compute probability and breakdown.
5. **Classify and notify.** High, Medium, Info or none; alerts to both units for High and Medium.
6. **Coordination.** Officers link, nominate a lead, hold parallel action or dismiss.
7. **Action tracking.** Every step logged: officer, unit, timestamp, remarks, documents, transfers, escalations.
8. **SLA monitoring.** Scheduled scan raises risk, breach and escalation alerts.
9. **Disposal.** Case closed with outcome; for linked cases the lead confirms consistent outcomes.
10. **Reporting.** Dashboards and exports for Bureau, region, institution, ageing, disposal and overlaps.

7.2 Roles

Role	Sees	Can do
Officer (RB, PGRC, other Bureau)	All cases for institutions they touch; overlap alerts	Register cases, log actions, decide overlaps
Regional Office	Their region's institutions and cases	Same, scoped to region
Management / Competent Authority	Everything; analytics; escalated cases; egress ledger	Final decisions on conflicts
Admin	Configuration, model card, audit log	Users, categories, SLA rules, thresholds, online/offline policy

8. Architecture and data pipelines

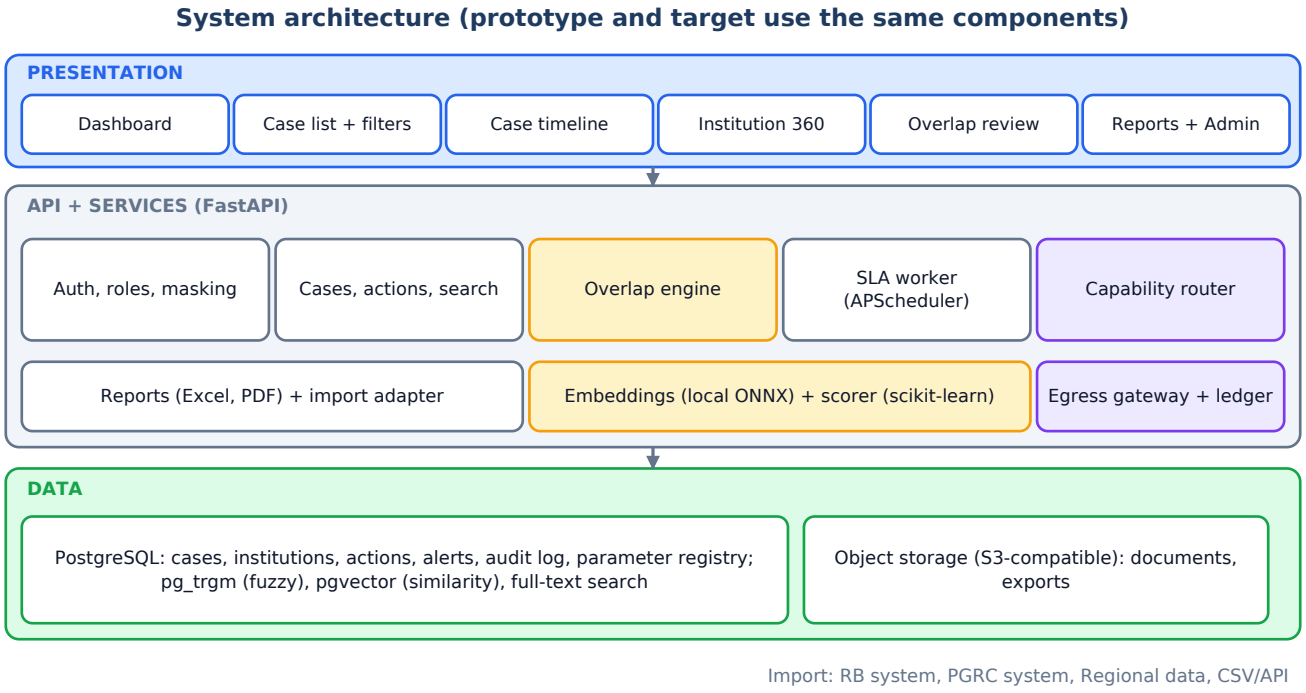


Figure 5. Layered architecture. The same containers run on a laptop, a single server, or a cloud account.

8.1 The pipelines

Pipeline	Trigger	Steps	Output	Main tools
Ingestion	New complaint or import file	Validate fields; resolve institution; assign category, SLA date, officer	Case + first action row	FastAPI, pg_trgm, Pydantic
Embedding	Case created or description edited	Embed text locally; store vector and model id	Vector on the case row	fastembed, pgvector
Overlap scan	After embedding	Fetch same-institution cases; build features; score; classify; write link and alerts	Overlap row + alerts	scikit-learn, SQL
SLA scan	Every 15 minutes (configurable)	Compute age, overdue days, breach risk; create alerts once per day per case	SLA risk, breach, escalation alerts	APScheduler
Retraining	Scheduled; or when enough new decisions exist	Build pairs from confirm/dismiss labels; train; evaluate; store as 'shadow'	New model version	scikit-learn
Promotion	After shadow review	Compare disagreements; approve; flip active model; keep rollback	Active model	Admin screen
Reporting	On request; scheduled exports	Aggregate by Bureau, region, institution, age, month	Charts, Excel, PDF	SQL, openpyxl, ReportLab
Audit	Every sensitive action	Append who, what, when, before/after	Audit rows	PostgreSQL

9. Online / Offline mode and privacy

The system keeps both an **online** and an **offline** provider for each optional capability and routes each request at runtime. Both are present at once; nothing is swapped at deployment.

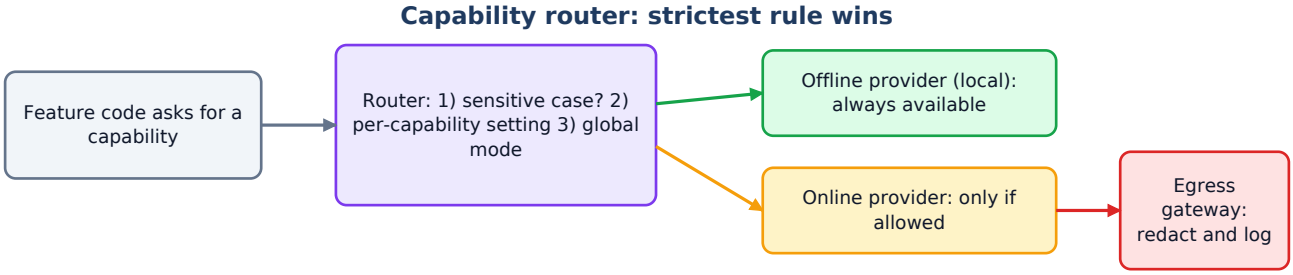


Figure 6. Router and egress gateway.

Capability	Online provider	Offline provider
Email alerts	Hosted email service	In-app alerts only (always on)
Overlap explanation text	Optional hosted language model [OPEN]	Template built from the score breakdown (always available)
Embeddings and scoring	Local model	Local model (identical in both modes)
Documents	Object storage	Same object storage; no external hand-off
Frontend assets, fonts, maps	Bundled	Bundled (never loaded from a CDN)

Three switches, strictest wins: (1) the case's sensitive flag, (2) a per-capability setting, (3) the global mode. Changes apply at runtime without restart and are audit-logged. If an online provider fails, the system falls back to the local one and tells the user; it never silently goes online.

What 'offline' enforces in the prototype: the application-level router and egress gateway block calls and record every attempt in an egress ledger; in the container demo the backend sits on an internal Docker network with no outbound internet. **What it does not claim:** cloud-network (VPC) controls and single-sign-on are production roadmap items.

> **Storage decision recorded:** there is one PostgreSQL connection string and one object-storage interface for both modes. Development runs PostgreSQL in Docker; the target deployment can use a managed PostgreSQL and S3 with configuration changes only. Do not describe a separate offline database.

10. Technology stack and why

Layer	Tool	Why we chose it	Alternative we rejected
Frontend	React + TypeScript, Vite	Components suit dashboards, tables, timelines; types catch data-shape bugs	Angular (heavier), Vue (smaller local talent pool)
	Tailwind CSS + shadcn/ui	Polished, accessible components we own	MUI/Ant (generic look)
	TanStack Query and Table	Auto-refresh gives the 'real-time' feel; powerful tables	Redux (overkill for server data)
	Recharts	KPI, workload, ageing, trend charts	D3 (too low-level)
Backend	Python + FastAPI + Pydantic	Async, auto OpenAPI docs, Python hosts the ML stack	Node (weaker ML), Django (heavier)
	SQLAlchemy, Alembic	Mature PostgreSQL support, migrations	Raw SQL only
	JWT + bcrypt	Stateless role-based auth	Server sessions
Database	PostgreSQL	Relational, linked data; transactions	MongoDB (poor fit for linked data)
	pg_trgm	Fuzzy name search inside the database	Separate search service
	pgvector (HNSW/IVFFlat, cosine)	Vector similarity in SQL (ref 9)	Dedicated vector database
	Object storage (S3 API)	Documents; any S3-compatible store works	Files inside the database
AI	fastembed + MiniLM multilingual	CPU-only, 384-dim, 50 languages, Apache-2.0 (refs 6, 8)	Full PyTorch stack; hosted LLM for matching
	scikit-learn	Logistic regression, calibration, metrics (ref 7)	Deep models (unexplainable, data-hungry)
	Splink (planned upgrade)	Fellegi-Sunter linkage, unsupervised, MIT (ref 4)	Hand-set weights
Jobs	APScheduler	Our recurring jobs are light and periodic	Celery + Redis/RabbitMQ (extra components, no benefit at this scale)
Packaging	Docker, Docker Compose	Same containers everywhere	Manual installs; Kubernetes (too heavy)
Quality	pytest, Vitest, Ruff, ESLint	Tests and code quality	-

Verified on 23 Oct 2026: the fastembed supported-models table lists sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 with 384 dimensions and an Apache-2.0 licence, so the schema's vector(384) is correct. Still run `TextEmbedding.list_supported_models()` after you install, because versions change.

11. Data model

Entity	Key fields	Notes
institutions	aicte_permanent_id, name, normalized_name, city, state, region	Trigram index on normalized_name
cases	institution_id, category, description, received_date, status, bureau_id, officer_id, sla_due_date, sensitivity_flag, hold_parallel, embedding vector(384), embedding_model	Vector and full-text indexes
actions	case_id, bureau_id, officer_id, action_type, remarks, meta, created_at	The timeline
documents	action_id, file_name, storage_key, checksum, storage_kind	Files live in object storage
overlap_links	case_a, case_b, level, probability, breakdown, status, lead_bureau_id, decision_reason, explanation_src, model_version	Unique pair; training labels
alerts	kind, case_id, overlap_id, bureau_id, message, read	Overlap, pending, escalation, SLA
bureaus, officers, categories, sla_rules	Org structure, access, deadlines with source_reference	Deadlines are policy
model_versions, parameter_registry	metrics, status (shadow/active/retired); parameter, value, origin, derived_from	The 'model card'
policy_settings, egress_ledger, audit_log	scope/value; capability/destination/outcome; actor/action/entity	Governance

Full SQL for every table is in the prototype build guide (Stage 1).

12. Prototype plan

The plan is organised in **stages**. Each stage ends with a short 'definition of done' check; do not start the next stage until it passes, and tag the Git commit so you can always return to a working demo. Sizes are relative (S, M, L), not time estimates, because team size and available time are not yet fixed.

Stage	Name	Output	Size
0	Tools and repo	Docker, Python, Node, Git; empty repository	S
1	Database	PostgreSQL in Docker with the full schema	S
2	Seed data	About 60 institutions, 400 cases, planted overlaps and hard negatives	M
3	Backend core	Login, cases, institutions, actions, dashboard statistics	M
4	Frontend shell	Login, layout, KPI cards, charts	M
5	Cases and institution pages	Filters, case detail with timeline, institution 360	M
6	Overlap engine	Resolution, embeddings, calibrated scorer, scan on case creation	L
7	Coordination and alerts	Review screen, link/lead/hold/dismiss, alerts center	M
8	SLA worker and reports	Risk scan, escalation, Excel/PDF export	M
9	Mode router and egress	Online/Offline toggle, sensitive flag, ledger, tests	M
10	Evaluation and registry	Metrics vs baseline, model card, shadow mode	M
11	Optional hosted demo	Short-lived cloud demo with a strict budget and teardown	S
12	Demo and PPT	Rehearsed demo, final PDF	S

12.1 Suggested split for four people (merge roles for a smaller team)

Person	Owns	Stages
A: Frontend lead	All pages, charts, overlap review card, mode bar	4, 5, UI of 7 and 9
B: Backend and database	Schema, API, auth, SLA worker, reports, router and ledger	1, 3, 7 (API), 8, 9
C: ML and matching	Seed data, resolution, embeddings, training, evaluation, model card	2, 6, 10
D: DevOps, QA, docs and PPT	Tools and repo, tests, hosted demo, demo script, final PDF	0, 11, 12, tests everywhere

12.2 Working agreements

- One Git branch per stage; merge to main only when its definition of done passes; tag each finished stage.
- Nobody changes the database schema alone: edit db/schema.sql, tell everyone, rebuild (docker compose down -v, then up).
- The API contract is the FastAPI /docs page; the frontend builds against it.
- Seed data is synthetic and says so; never present it as real.
- Keep a running list of 'claims we can defend' for the PPT (Section 16).

12.3 Local setup in five commands

```
git clone <repo> && cd aicte-dashboard
docker compose up -d db                      # PostgreSQL + pgvector
cd backend && python -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
python -m scripts.seed && python -m ml.train_overlap
uvicorn app.main:app --reload     # then, in another terminal: cd frontend && npm install && npm run dev
```

The hosted demo is optional; check whether your SIH round needs a live link before spending anything. A laptop demo plus a short screen recording is often enough.

13. Evaluation plan

Judges trust numbers they can inspect. We measure four things and report them honestly.

What	How	Report
Overlap detection quality	Labelled case pairs (true overlap = same institution and same underlying issue); split by institution; calibrated model	Precision and recall at High and Medium thresholds, PR-AUC, example false positives/negatives
Versus a baseline	Compare against 'cosine similarity above a fixed cutoff' alone	Same metrics side by side: shows our features add value
Institution matching	About 100 messy name variants labelled correct/incorrect	Auto-accept precision, review rate, recall
Alert volume	Count High and Medium alerts per active case per day	Evidence of manageable load
SLA warning	Replay closed cases	How early the warning arrives and how many false warnings

Synthetic data design: templates per complaint issue with several paraphrases (including a few Hinglish ones); planted true overlaps handled by RB and PGRC; **hard negatives** (same institution, different issue); messy institution name variants; closed-case durations drawn from a skewed distribution so SLA statistics exist.

> **Reporting rules:** say 'synthetic'; show the baseline; show failures; never quote a number we did not measure; keep the evaluation script in the repository so anyone can re-run it.

14. Risks and open decisions

Risk	Likelihood	Mitigation
Synthetic data makes results look too good	High	Hard negatives, baseline comparison, inspect errors, state it openly
Too many features, weak story	High	Protect three things: overlap alert and coordination, institution 360, live dashboard
Embedding model too slow or too big for the host	Medium	CPU ONNX model; load once; test memory early; model name is configurable
False alerts annoy officers	Medium	Precision-targeted High threshold; dismiss-with-reason; alert volume check
Overclaiming privacy or integration	Medium	Use the wording in Section 16; list what is roadmap
Library or cloud details change	Medium	Pin versions after it works; re-check pricing and extension support before any hosted step
Team capacity	Unknown	Stage plan lets us stop after any 'definition of done' with a working demo

14.1 Open decisions for the team meeting

#	Decision	Suggestion
1	Per-case sensitive flag that forces offline processing	Yes; it is cheap and a strong privacy story
2	'AUTO' routing mode (prefers offline, uses online only when needed)	Skip unless time remains
3	Optional hosted language-model explanations	Roadmap slide only; template explanations in the build
4	Institution risk score and India region map	Treat as stretch; build only after Stage 10
5	Team size, roles and available time	Needed to fix the stage plan
6	Does the round require a live link or accept a video?	Check the SIH instructions before any hosting work
7	Exact theme text, team ID and team name for the title slide	Copy from the SIH portal
8	Final project name	CaseNexus is a placeholder

15. Seven-minute demo script

Time	Step	What to show
0:00	The problem	Institution with active RB and PGRC cases; neither knows
0:45	Dashboard	Live KPIs, Bureau workload, the overlap banner
1:30	The moment	Register an RB complaint worded differently; High overlap fires with the explanation card
3:30	Coordination	Link, nominate lead, hold the other case; both timelines update
4:30	Precision	A same-institution, different-issue case is not flagged High
5:15	SLA and reports	Risk alert, ageing chart, Excel export
6:00	Privacy	Switch to Offline, mark a case sensitive, show the empty egress ledger
6:30	Evidence	Precision and recall versus the baseline [fill after measuring]

Rehearsal tips: reset the data before every run (Section 12.3); load the embedding model before the demo so the first request is not slow; keep a two-minute screen recording as a fallback.

16. Submitting the idea PPT

The SIH template (attached to the project) limits the idea PPT to **six slides including the title slide**, asks for points and diagrams rather than paragraphs, forbids changing its pointer headings, and requires upload as a **PDF**. The paste-ready text is in *SIH2026_PPT_Content.md*.

Slide	Template heading	What goes in
1	Title page	PS ID 26264, PS title, theme, category Software, team ID, team name
2	Idea title + Proposed solution	Detailed explanation; how it addresses the problem; innovation and uniqueness
3	Technical approach	Technologies; methodology and process flow chart; prototype
4	Feasibility and viability	Feasibility; challenges and risks; strategies
5	Impact and benefits	Impact on target audience; social, economic, environmental benefits
6	Research and references	Verified links (Section 18)

16.1 Claims we can defend, and claims we cannot

Safe to say	Do not say
Prototype on synthetic data demonstrates the full flow	'Tested on real AICTE data'
Alerts are explainable and thresholds are learned and recorded	'95% accuracy' before we have measured it
Offline mode makes no external calls; the egress ledger proves it in the demo	'Fully secure' or 'compliant with government hosting rules'
Import adapter maps RB/PGRC data to one schema	'Integrated with AICTE's systems'
The system suggests; officers decide	'AI replaces officer decisions'
Rarity-weighted fuzzy matching built; Fellegi-Sunter via Splink planned	'Fellegi-Sunter implemented' unless it truly is

Never mention internal cloud-credit limits on slides.

17. Glossary

Term	Meaning
AICTE	All India Council for Technical Education, the regulator of technical education
RB / PGRC	Regulation Bureau / Public Grievance Redressal Cell
CPGRAMS	Centralised Public Grievance Redress and Monitoring System, the government's grievance portal
SLA	Service-level agreement: the prescribed time within which a case should be handled
Institution 360	One page showing every case about an institution from every unit
Entity resolution / record linkage	Deciding whether two records refer to the same real-world entity
Fellegi-Sunter	A statistical model that sums evidence from each field agreeing or disagreeing
EM algorithm	A method that estimates model parameters without labelled data
Trigram	A three-character piece of text, used to measure how alike two strings are
IDF	Inverse document frequency: a weight that makes rare words count more
Embedding	A list of numbers representing the meaning of a text
Cosine similarity	A measure of how closely two embeddings point in the same direction
pgvector	PostgreSQL extension that stores embeddings and searches by similarity
ONNX	A portable model format that runs fast on a CPU
Logistic regression	A simple model that turns weighted features into a probability
Calibration	Adjusting scores so they behave like real probabilities
Precision / recall	Share of alerts that are right / share of true overlaps found
PR-AUC	Area under the precision-recall curve; a summary for imbalanced problems
Hard negative	A look-alike pair that is NOT related, used to test false alarms
Alert fatigue	When too many alerts make users ignore all of them
Leakage	When test data secretly overlaps with training data, inflating results
Shadow mode	Running a new model silently beside the active one to compare before switching
RBAC	Role-based access control
Egress ledger	A log of every attempt to send data outside the system
Synthetic data	Realistic but artificially generated data used because real data is unavailable

18. References

All links below were opened or searched on 23 October 2026. Where a page was an older version, the note says so.

#	Reference	Link
1	AICTE, Grievance Redressal page (PGRC, categories of grievance, CPGRAMS grievances).	https://aicte.gov.in/node/422
2	Press Information Bureau, document on CPGRAMS (scale, 21-day timeline, appeals), December 2024.	https://static.pib.gov.in/WriteReadData/specificdocs/documents/2024/dec/doc20241230478501.pdf
3	Fellegi, I. P. and Sunter, A. B. (1969). A Theory for Record Linkage. Journal of the American Statistical Association, vol. 64.	https://pubs.dbs.uni-leipzig.de/dc/node/742

#	Reference	Link
4	Splink: fast, accurate probabilistic data linkage (Fellegi-Sunter implementation, unsupervised training, term-frequency adjustments, MIT). The page I read documented version 3.7.1; newer releases exist.	https://pypi.org/project/splink/3.7.1
5	Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv:1908.10084.	https://arxiv.org/abs/1908.10084
6	Model card: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 (384-dimensional vectors, 50 languages, Apache-2.0).	https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
7	scikit-learn documentation: CalibratedClassifierCV (sigmoid/Platt and isotonic guidance).	https://scikit-learn.org/dev/modules/generated/sklearn.calibration.CalibratedClassifierCV.html
8	FastEmbed (ONNX Runtime, CPU, Apache-2.0).	https://github.com/qdrant/fastembed
9	pgvector: open-source vector similarity search for Postgres (HNSW and IVFFlat; cosine distance operator).	https://github.com/pgvector/pgvector
10	FastEmbed supported models (lists the multilingual MiniLM model with 384 dimensions).	https://qdrant.tech/documentation/embeddings/fastembed/
11	Quantized ONNX port of the model for FastEmbed.	https://huggingface.co/Qdrant/paraphrase-multilingual-MiniLM-L12-v2-onnx-Q
12	Smart India Hackathon 2026 idea submission template (the attached PPTX; rules on slide limit and PDF upload).	Attached file: SIH2026-IDEA-Presentation-Format

Things not independently checked: AICTE's internal timelines and categories beyond the grievance page; the formats of RB/PGRC systems; any hosting or procurement rule for government deployments; library versions and cloud prices. Confirm these before relying on them.